

DSA 期末考前学案：校订详细版

严格依据错题 PDF、对应 Python 代码、tree.md 与 graph.md

根据当前文件夹资料重新校订

2026 年 6 月

目录

校订说明	3
1 两个 md 文件实际覆盖了什么	3
1.1 tree.md 的真实内容	3
1.2 graph.md 的真实内容	3
2 总体薄弱点：按实际代码归纳	4
2.1 薄弱点一：模板和题目输入之间的转换	4
2.2 薄弱点二：相似题名下算法目标不同	4
2.3 薄弱点三：手写结构和库函数的对应关系	4
3 树类题逐题校订复盘	5
3.1 22 根据二叉树前中序序列建树	5
3.2 23 括号嵌套二叉树	5
3.3 24 二叉树	5
3.4 25 二叉搜索树的层次遍历	6
3.5 26 实现堆结构	6
3.6 27 树的转换	7
3.7 28 Huffman 编码树	7
3.8 49 二叉树的深度	7
3.9 50 扩展二叉树	8
3.10 53 根据二叉树中后序序列建树	8
3.11 54 验证二叉搜索树	8
4 图类题逐题校订复盘	9
4.1 1 单词序列	9
4.2 2 仙岛求药	9
4.3 3 变换的迷宫	9
4.4 1 拓扑排序	10
4.5 2 丛林中的路	10
4.6 3 A Walk Through the Forest	10
4.7 4 打怪救公主	11
4.8 5 兔子与樱花	11
4.9 56 拯救行动	12
4.10 57 连接所有点的最小费用	12
5 横向对比：md 模板如何被你实际改动	12

6	graph.md 中有具体代码的算法清单与题目映射	13
6.1	graph.md 中实际给出代码的算法/模板	13
6.2	作业题目与 graph.md 代码算法之间的映射	14
7	考前落地检查表	15
8	最该优先复习的错题组合	15

校订说明

这份版本专门修正上一版的问题：不再把没有出现在 `tree.md` 或 `graph.md` 中的内容说成“md 里已有”，也不再把 PDF 中你实际采用的代码方法改写成另一套更短或更常见的做法。每道题都按三件事来写：

1. `md` 中确实相关的内容：只写两个 `md` 文件里实际出现的模板、概念或例子。
2. PDF/对应 Python 代码的实际做法：以当前 PDF 的 Listing 和同名 `.py` 文件为准。
3. 从模板到本题的迁移：解释你的代码为什么这样改，而不是另起炉灶。

需要警惕

特别校订的几处：实现堆结构.py 实际是手写 BinHeap，不是直接用 `heapq`；兔子与樱花.py 实际是图对象 + 无优先队列 Dijkstra，不是 Floyd；丛林中的路.py 实际是邻接矩阵 + Prim，不是 Kruskal；括号嵌套二叉树.py 实际是按下标递归解析，不是“最外层逗号 split”。这些都会按实际代码讲。

1 两个 md 文件实际覆盖了什么

1.1 tree.md 的真实内容

资料依据

`tree.md` 实际覆盖：

- 二叉树节点类、前序/中序/后序递归遍历。
- 二叉树广度优先遍历。
- 从层次遍历建树。
- 普通树按“左儿子右兄弟”转换成二叉树。
- 前序 + 中序重建二叉树。
- 非递归前序/中序遍历。
- 二叉堆 ADT、手写堆的上浮/下沉/buildHeap、堆排序。
- Python `heapq` 的最小堆用法。
- BST 的节点、插入、查找、删除。
- Huffman/最优二叉树：最小 WPL、真正构造 Huffman 树、Huffman 解码、木板切割最小费用。

它没有直接覆盖的作业细节包括：括号嵌套二叉树的按下标解析、扩展二叉树的，空节点输入、BST 合法性上下界验证、完全二叉树编号区间计数。这些属于作业/Python 文件中的具体题型拓展。

1.2 graph.md 的真实内容

资料依据

`graph.md` 实际覆盖：

- 图的基本概念、邻接矩阵、邻接列表、Vertex/Graph 类。
- 词梯问题：桶建图 + BFS。
- BFS 原理、距离/前驱/颜色、路径回溯。
- 数轴 BFS“抓住那头牛”。

- 普通迷宫 BFS，带 parent 的路径恢复。
- 迷宫变形：剩余消除障碍能力作为状态维度。
- 进入守卫等不同代价迷宫：普通 BFS 失效，改优先队列/Dijkstra。
- 更复杂钥匙/守卫状态设计的讨论。
- DFS、骑士周游、城堡问题、踩方格、算 24。
- 拓扑排序、AOE 关键路径、强连通分支。
- Dijkstra，包括优先队列版本和不用优先队列版本。
- Floyd，包括 dist 和 prev。
- Prim，包括对象图版本、heapq 版本、邻接矩阵版本。
- Kruskal 的文字定义。

它没有直接覆盖的作业细节包括：A Walk Through the Forest 的“从 2 跑 Dijkstra 后按距离下降 DFS 计数”、兔子与樱花中你实际用“每次查询重新跑无优先队列 Dijkstra”、连接所有点的最小费用中你实际先建完全图再 Prim。这些必须以 Python 文件为准讲解。

2 总体薄弱点：按实际代码归纳

2.1 薄弱点一：模板和题目输入之间的转换

你的很多代码是从 md 模板改出来的，但题目输入形式发生了变化：

- tree.md 中前序 + 中序重建是模板；作业 22 需要多组 EOF，并在建树后后序输出。
- tree.md 中层次建树是模板；作业 54 用它先建树，再做 BST 上下界验证。
- graph.md 中词梯是从文件读单词；作业“单词序列”从一行输入读 wordlist，还要把 start/end 加进去。
- graph.md 中 Dijkstra 是对象图模板；兔子与樱花直接使用这个对象图结构，**但每次查询前要 reset。**

2.2 薄弱点二：相似题名下算法目标不同

不要只看题目像不像，而要看答案目标：

- 仙岛求药：普通 BFS，目标是最短步数。
- 打怪救公主：不是最短步数，而是最大剩余血量，所以用 max_blood 二维数组。
- 拯救行动：最短时间，但边权 1/2 不同，所以用 Dijkstra。
- A Walk Through the Forest：最终是路线数量，不是最短路长度；Dijkstra 只是中间步骤。

2.3 薄弱点三：手写结构和库函数的对应关系

本次代码里既有手写结构，也有库函数：

- 实现堆结构：手写 BinHeap，与 tree.md 的手写堆最接近。
- Huffman：使用 heapq，与 tree.md 的最小 WPL 代码最接近。
- 拓扑排序、拯救行动、Prim：都使用 heapq，但含义不同。拓扑堆存点编号，Dijkstra 堆存距离，Prim 堆存连接边权。

3 树类题逐题校订复盘

3.1 22 根据二叉树前中序序列建树

资料依据

md 依据: tree.md 中“前序 + 中序重建二叉树”的基本原则和 Buildtree(preorder, inorder) 代码。

本题实际代码方法

你的实际代码: 定义 Treenode; 用 Buildtree(preorder, inorder) 递归建树; 再用 postorder(root, ans) 输出后序; 主程序用 while True + try/except EOFError 处理多组输入。

代码建立思路: 这题最直接依赖 tree.md 里的“前序 + 中序重建二叉树”模板。现成部分是: 前序第一个元素当根, 在中序中找到根的位置, 并据此切出左、右子树。你新写/修改的部分是两个: 第一, 建好树后接上后序遍历函数, 因为题目要输出后序; 第二, 主程序改成 EOF 多组输入。组合方式就是: 读一组前序/中序 → 用 md 模板建树 → 后序遍历收集字符 → 输出。

迁移关系:

1. md 里已经有前序根 preorder[0] 和中序切左右。
2. 本题多了“输出后序”, 所以不是建完就结束, 而是后序遍历收集答案。
3. 本题多组到 EOF, 所以主循环不是只读一次。

易错点: 前序左子树切片必须用左子树长度 $l = \text{len}(\text{ino_left})$, 你的代码正是 `preorder[1:1+l]`。

3.2 23 括号嵌套二叉树

资料依据

md 依据: 只有二叉树节点和前序/中序遍历模板; tree.md 没有直接给括号嵌套解析模板。

本题实际代码方法

你的实际代码: `buildtree(s, i)` 按字符串下标递归解析。遇到 * 返回空和新下标; 读到根字符后, 如果后面是 (, 递归读左子树, 遇逗号后递归读右子树, 遇) 返回。

代码建立思路: 这题只借 tree.md 的二叉树节点类和递归遍历思想, 解析括号串这一块是本题新写的。你先保留“一个节点有 left/right”的树结构, 再把输入字符串看成递归语法: 当前字符是根, * 是空树, 括号里依次是左子树和右子树。新写的关键是让 buildtree 返回“子树根 + 读到的新下标”, 这样递归解析和字符串位置同步推进; 建完后再接上 tree.md 里熟悉的前序/中序输出。

这题最重要的是“函数返回两个东西”: 子树根节点和当前解析到的新下标。这样避免了自己手动找最外层逗号。

```
root.left, i = buildtree(s, i)
if i < len(s) and s[i] == ",":
    i += 1
    root.right, i = buildtree(s, i)
```

复习时要把它归到“树遍历 + 字符串递归解析”, 不要误以为 md 已经有完整解析模板。

3.3 24 二叉树

资料依据

md 依据: tree.md 中完全二叉树编号性质: 左孩子 $2p$, 右孩子 $2p+1$ 。

本题实际代码方法

你的实际代码：不建树。读入 n, m ，其中 n 是子树根编号， m 是总编号上界；用 $left/right$ 表示当前层在这棵树中的编号区间，逐层扩张并截断。

代码建立思路：这题依赖 `tree.md` 中完全二叉树的编号关系，而不是依赖节点建树模板。现成概念是：编号为 p 的节点，左孩子是 $2p$ ，右孩子是 $2p+1$ 。你新写的做法是把“一个子树在每一层上的所有编号”压缩成区间 $[left, right]$ ：下一层区间变为 $[left*2, right*2+1]$ 。每层只统计不超过 m 的编号个数，所以整题组合为：根编号区间初始化 → 按完全树编号扩展 → 每层截断计数。

关键公式：

```
ans += min(m, right) - left + 1
right = right * 2 + 1
left = left * 2
```

这是从 `md` 的“编号性质”进一步变成“区间计数”。这里不是 `md` 里直接给出的题型，但完全依赖那个性质。

3.4 25 二叉搜索树的层次遍历

资料依据

`md` 依据：BST 插入、二叉树 BFS/层次遍历。

本题实际代码方法

你的实际代码：手写 `TreeNode` 和 `BinarySearchTree.put`；插入时小于走左，大于走右，相等不处理；最后 `bfs(root, ans)` 用列表队列做层次遍历。

代码建立思路：这题是 `tree.md` 里 BST 插入模板和二叉树层次遍历模板的组合。第一段沿用 BST 的比较逻辑：从根开始，小于当前节点就往左，大于当前节点就往右，直到空位置插入；作业中特别修改的是相等元素不插入。第二段沿用层次遍历：把根放入队列，弹出节点就记录值，并把非空左右孩子入队。组合方式就是：输入序列逐个插入成 BST → 对生成的 BST 做 BFS → 输出层序结果。

本题从 `md` 到作业的变化是：不是单独考 BST，也不是单独考 BFS，而是“先用输入序列构造 BST，再按层次输出”。重复值忽略是作业题面特有点，`md` 的通用 BST 插入代码没有强调这个输出要求。

3.5 26 实现堆结构

资料依据

`md` 依据：`tree.md` 中手写 `BinHeap` 的初始化、`percUp`、`percDown`、`delMin`。

本题实际代码方法

你的实际代码：使用手写 `BinHeap`，不是 `heapq`。每组测试新建一个堆，操作 1 调 `insert`，操作 2 调 `delMin` 并打印。

代码建立思路：这题直接依赖 `tree.md` 的手写二叉堆 ADT，而不是 Python 库堆。现成部分是用数组保存完全二叉树、用下标关系找父子节点、插入后 `percUp`、删除堆顶后 `percDown`。你新写/修改的部分是把比较方向统一成小顶堆，并把堆操作包进题目的操作流：操作 1 读一个数插入，操作 2 删除并输出当前最小值。组合方式就是：每组数据初始化空堆 → 按操作调用堆方法 → 删除时打印返回值。

需要特别注意：`tree.md` 里手写 `percUp` 的比较方向写成了大顶堆倾向，而你的作业代码改成 `<`，实现的是最小堆。这是从笔记到题目要求的关键修正。

```
if self.heaplist[i] < self.heaplist[i//2]:
    self.heaplist[i], self.heaplist[i//2] = ...
```

3.6 27 树的转换

资料依据

md 依据：普通树转左儿子右兄弟二叉树的 `convert(root)` 模板。

本题实际代码方法

你的实际代码：先根据 `d/u` 字符串真正建一棵普通树，节点有 `children` 和 `parent`；再调用 `convert` 生成二叉树；最后分别用 `length(root)` 和 `height(b_root)` 输出原树高度和二叉树高度。

代码建立思路：这题依赖 `tree.md` 的“普通树转左儿子右兄弟二叉树”模板，但题目没有直接给一棵现成的普通树，所以你先新写了建普通树的过程。读到 `d` 就创建新孩子并下移，读到 `u` 就回到父节点；这样得到带 `children` 列表的普通树。接着套 `md` 的转换思想：第一个孩子变成二叉树左孩子，后续兄弟沿右指针串起来。最后再分别写两个高度函数，原树高度按 `children` 递归，转换后的二叉树高度按 `left/right` 递归。

这题不是“只模拟高度”的写法，而是与 `md` 中转换模板高度一致：先建普通树，再转换，再递归求高度。注意你的 `height(None)` 返回 `-1`，所以叶子二叉树高度为 `0`；`length` 也按边数口径处理。

3.7 28 Huffman 编码树

资料依据

md 依据：`tree.md` 中“计算最小 WPL”的 `heapq` 版本和最优二叉树构造思想。

本题实际代码方法

你的实际代码：`optimal_binary_tree(weights)` 直接 `heapq.heapify`，每次弹出两个最小值，合并值加入 `total_wpl`，再压回堆。主程序按测试组读取。

代码建立思路：这题依赖 `tree.md` 中 Huffman/最优二叉树的“最小 WPL”代码，而不是依赖真正输出编码的树结构。现成部分是 `heapq` 小根堆：每次取出两个最小权值合并，合并代价加入总 WPL，再把新权值压回。你新写/修改的部分主要是读多组测试数据，以及把函数命名为 `optimal_binary_tree` 只返回最小带权路径长度。组合方式就是：权值列表建堆 → 反复两两合并 → 累加所有合并代价。

这里和 `md` 的最小 WPL 代码基本一致。不要把它讲成“必须真正构造 Huffman 树”；你的作业代码只计算 WPL。

3.8 49 二叉树的深度

资料依据

md 依据：二叉树节点和递归高度思想。

本题实际代码方法

你的实际代码：先创建 `nodes=[None]*(n+1)`，为 `1` 到 `n` 每个编号建 `BinaryNode`；再读每个节点的左右孩子编号并连接对象；最后 `height(nodes[1])` 输出深度。

代码建立思路：这题依赖 `tree.md` 的二叉树节点类和递归求高度思想。现成部分是“节点有左孩子、右孩子，树高由左右子树高度递归得到”；你新写的部分是把题目的编号输入转换成真实节点连接。代码先为每个编号预建对象，再根据输入的左右孩子编号把 `left/right` 指针接好，编号 `-1` 表示空。组合方式就是：编号表建节点 → 按输入连边 → 从 `1` 号根递归算高度/深度。

和更短的数组递归不同，你的代码走的是“编号输入转节点对象”的路线。这与 `tree.md` 中用类表示树节点的思路一致。

3.9 50 扩展二叉树

资料依据

md 依据：二叉树节点、递归遍历。md 没有直接给扩展先序，输入模板。

本题实际代码方法

你的实际代码：build_tree(seq) 内部定义 helper()，用 nonlocal idx 顺序读先序串；遇到 . 返回 None；否则建节点并递归建左、右子树。最后输出中序和后序。

代码建立思路：这题依赖 tree.md 的二叉树递归遍历，但建树方式是作业新加的“扩展先序”。现成部分是节点结构和中序/后序遍历函数；新写部分是 helper() 顺序消费字符串：每读到一个普通字符就建节点，然后递归读它的左子树和右子树；每读到 . 就返回空节点。组合方式就是：扩展先序串 → 用全局/闭包下标递归建树 → 对建出的树分别中序、后序输出。

本题核心是“带空标记的先序能唯一建树”。从 md 的遍历模板迁移时，新增的是顺序读入下标和空节点出口。

3.10 53 根据二叉树中后序序列建树

资料依据

md 依据：前序 + 中序重建的切分思想。md 没有直接给中序 + 后序代码。

本题实际代码方法

你的实际代码：Buildtree(postorder, inorder)；根是 postorder[-1]；用中序切左右；按左子树长度切后序；建树后前序输出。

代码建立思路：这题依赖 tree.md 中“用中序确定左右子树范围”的重建思想，但把前序模板改成了后序版本。现成逻辑是：必须在中序里找到根，根左边是左子树，根右边是右子树。你修改的关键是根不再取 preorder[0]，而是取 postorder[-1]；切后序时要排除最后的根，并按左子树长度拆成左右两段。组合方式就是：中序定位根 → 切左右子树序列 → 递归建树 → 前序遍历输出。

这是第 22 题的对称迁移：根的位置从前序开头换成后序末尾，输出方式从后序换成前序。

3.11 54 验证二叉搜索树

资料依据

md 依据：层次遍历建树模板、BST 的基本性质。md 没有直接给“验证 BST 上下界递归”。

本题实际代码方法

你的实际代码：先用 build_tree_level(level_list) 从层序输入建树，# 表示空；再用 is_bst(node, low, high) 递归验证，每个节点必须满足 $low < val < high$ 。

代码建立思路：这题分两段，第一段依赖 tree.md 的层次遍历建树模板：用队列按顺序给当前节点接左、右孩子，遇到 # 就不建节点。第二段依赖 BST 的性质，但验证函数是本题新写的增强版：不能只比较父子大小，而是把祖先给出的上下界 low/high 一路传下去。组合方式就是：层序字符串建出普通二叉树 → 从根开始做上下界递归 → 左子树收紧上界，右子树收紧下界。

这题的拓展点在于：不能只看一个节点和左右孩子的大小关系，而要把祖先限制一路传下去。你的代码已经用上下界处理了这个问题。

4 图类题逐题校订复盘

4.1 1 单词序列

资料依据

md 依据: graph.md 词梯问题中的桶建图、Vertex/Graph 类、BFS 的 color/distance/pred。

本题实际代码方法

你的实际代码: 几乎沿用 md 的对象图结构; buildGraph(wordlist) 用通配桶把只差一个字母的单词连边; BFS 用 deque, 设置 color、distance、pred; 答案输出 e.getDistance()+1, 不可达输出 0。

代码建立思路: 先直接拿 graph.md 里的 Vertex/Graph 邻接表类, 作为存单词图的骨架; 再拿词梯的“通配桶建图”代码, 把输入方式从读文件改成读一行 wordlist, 并把起点、终点补进词表。建完图后接上 md 的 BFS 模板, 用 distance 表示从起点到某个单词需要换几次; 最后新写的部分主要是输入输出和答案转换: 如果终点没被访问输出 0, 否则输出边数距离加 1, 表示序列中单词个数。

从 md 到作业的具体改动:

- md 是从文件读单词, 本题从一行 wordlist=input().split() 读。
- 你的代码把 start 和 end 也 append 到 wordlist, 避免起终点不在字典中。
- 输出的是单词序列长度, 所以边数距离要加 1。

4.2 2 仙岛求药

资料依据

md 依据: 普通迷宫 BFS, 二维 visited, 四方向扩展。

本题实际代码方法

你的实际代码: bfs(maze,M,N,sx,sy) 从 @ 出发, 队列元素为 (x,y,dis); 遇到 * 返回距离; # 不可走; 0 0 结束。

代码建立思路: 这里直接用 graph.md 的普通迷宫 BFS 模板, 不需要建显式 Graph 类, 因为网格本身就是隐式图。保留模板中的队列、四方向数组、二维 visited; 新写/修改的是读多组输入、扫描 @ 起点、把目标判断改成 maze[nx][ny]=='*', 并把墙判断固定为 # 不能走。组合顺序就是: 读迷宫 → 找起点 → BFS → 返回最先到达终点的步数。

这题和 md 普通迷宫模板最接近。注意你的起始距离是 0, 因此输出的是移动步数, 不是包含起点的格子数。

4.3 3 变换的迷宫

资料依据

md 依据: 迷宫变形问题中“同一个位置在不同状态下不同, 要用三维 visited”的思想。

本题实际代码方法

你的实际代码: visited[M][N][K], 状态是位置加时间模数; 如果下一格是 # 且 (dis+1)%K != 0, 则不能进入; 到达 E 返回时间, 不可达输出 0op!。

代码建立思路: 这题借的是 graph.md 里“状态 BFS”的搭法, 而不是照搬剩余能力的含义。现成部分是队列 BFS、四方向移动、visited 升成三维; 修改部分是把第三维从“剩余能力”换成“时间模数 t%K”。新写的关键

判断是：普通格可以按 BFS 进入，墙 # 只有在下一时刻 $(dis+1)\%K==0$ 才能通过。代码组合为：网格位置 + 当前时间共同定义状态，入队时同步更新 $dis+1$ 和时间模数。

这里与 md 的“剩余消除能力 k ”不完全相同：md 的第三维是剩余能力，本题第三维是时间模数。但迁移原则一致：二维位置不够表示状态，必须升维。

4.4 1 拓扑排序

资料依据

md 依据：拓扑排序的入度数组 + 队列版本。

本题实际代码方法

你的实际代码：用 `heapq` 维护当前入度为 0 的点，实现“编号小优先”；图数组 $G=[[] \text{ for } _ \text{ in range}(n+1)]$ ，顶点从 1 到 n ；输出格式为 `v1 v2 ...`。

代码建立思路：主体来自 `graph.md` 的拓扑排序：建邻接表、统计入度、不断取入度为 0 的点、删除它的出边。你修改的核心只有一个：把 md 里的普通队列换成 `heapq`，这样多个点同时入度为 0 时会先弹出编号最小者。其他新写部分主要是作业格式适配：顶点编号从 1 开始，读入边后更新 `indegree[v]`，最后把结果加上 `v` 前缀输出。

从 md 到作业的变化：md 讲的是普通队列输出任一拓扑序；你的代码为了编号小优先，把队列换成了最小堆。

4.5 2 丛林中的路

资料依据

md 依据：Prim 的邻接矩阵版本，以及 MST 的定义。md 也有 Kruskal 文字，但你的代码没有用 Kruskal。

本题实际代码方法

你的实际代码：`Graph_adj_mat` 存邻接矩阵；输入字母用 `ord(ch)-ord('A')` 转编号；无向边双向写入矩阵；调用 `prim(G,0)`，堆中存（边权，顶点编号）。

代码建立思路：这里使用的是 `graph.md` 的 Prim 邻接矩阵版本。现成部分是邻接矩阵图结构和 Prim 的“从已选集合向外找最小边”逻辑；新写/修改部分是输入解析，因为题目用大写字母表示点，还按每个点后面跟若干条边的形式给数据。你的代码先把字母转成 0 开始的编号，再把每条无向边同时写入 `G[u][v]` 和 `G[v][u]`，最后把矩阵图交给 `prim(G,0)` 汇总 MST 边权。

这题必须校订：它不是 Kruskal。复习时应和 `graph.md` 中“基于邻接矩阵的 Prim”对应起来。

4.6 3 A Walk Through the Forest

资料依据

md 依据：Dijkstra 对象图模板、`heapq` 过期状态处理、DFS/记忆化思想可从图搜索部分迁移。md 没有直接给这道题的组合。

本题实际代码方法

你的实际代码：建 $1..n$ 的对象图，读无向带权边；从顶点 2 跑 Dijkstra，得到每个点到家的最短距离；然后 `dfs_count_routes(u,memo)` 从 1 开始计数，只能走向 `v.distance < u.distance` 的邻居。

代码建立思路：这题是组合题。第一段借 graph.md 的对象图 + Dijkstra 优先队列版：用 Vertex/Graph 存无向带权边，并从终点 2 跑最短路，得到“每个点离家有多近”。第二段不是 md 的原样代码，而是你新写的 DFS 计数：从 1 出发，只允许走向 distance 更小的邻居，因为题目要求每一步都更接近家。最后用 memo 记忆化每个点到 2 的合法路线数，把 Dijkstra 的距离结果和 DFS 的路线计数组合起来。

这题要特别分清两层：

1. Dijkstra 的答案不是最终输出，而是为“更接近家”提供判断依据。
2. DFS 的终点是编号 2，memo[u] 记录从 u 到家的合法路线数。

4.7 4 打怪救公主

资料依据

md 依据：普通迷宫 BFS、以及“状态中资源会影响后续”的思想。md 没有直接给最大剩余血量版本。

本题实际代码方法

你的实际代码：max_blood[x][y] 存到达每个格子的最大剩余血量；队列元素是 (x,y,blood)；进入数字格扣血，扣到 0 或以下不能继续；如果 new_blood > max_blood[nx][ny] 才入队。

代码建立思路：基础移动框架来自 graph.md 的普通迷宫 BFS：队列、四方向、边界判断、墙/可走格判断。你新写的关键是把 visited 从布尔数组改成 max_blood，因为同一个格子“到过”不够，还要看剩余血量是否更优。每次扩展时先按格子内容计算 new_blood，如果血量没有超过该格历史最好值就丢弃；如果更高，才更新数组并入队。也就是说：BFS 的壳保留，判重条件从“没访问过”改成“资源状态更好”。

这题不是求最短路，所以不能用普通 visited=True/False。你的代码正确保留了“更高血量重访同一格”的可能。

4.8 5 兔子与樱花

资料依据

md 依据：graph.md 中 Dijkstra 不用优先队列版本、Vertex/Graph 类、前驱 pred 恢复路径。md 的 Floyd 部分不是你这题实际采用的方法。

本题实际代码方法

你的实际代码：地点名映射到 Vertex；道路是无向边；每次查询时对起点运行 dijkstra_no_pq，通过 pred 从终点回溯路径，再按 A->(w)->B 输出。

代码建立思路：这题直接沿用 graph.md 的 Vertex/Graph 对象图，以及无优先队列 Dijkstra 的主体：每轮扫描未访问点中当前距离最小的点，再松弛它的邻居。你新写/修改的部分有三块：第一，把城市字符串映射成顶点对象，而不是用数字编号；第二，因为有多次查询，每次跑 Dijkstra 前都要 reset 所有顶点的 distance/pred/visited；第三，Dijkstra 结束后不是只输出距离，而是沿 pred 从终点回溯路径，再根据相邻城市查边权，格式化成果目要求的路径字符串。

这题必须校订：它不是 Floyd。虽然 Floyd 也适合多查询，但你的代码实际选择了“每次查询跑一次无优先队列 Dijkstra”。讲解时应重点解释 reset、visited 集合、pred 路径恢复。

4.9 56 拯救行动

资料依据

md 依据：graph.md 中“进入守卫格代价不同，所以普通 BFS 转 Dijkstra”的迷宫例子。

本题实际代码方法

你的实际代码：每组读迷宫，找到 r 和 a；dist[x][y] 存最短时间；堆中存 (steps,x,y)；进入 x 代价 2，否则代价 1；遇到终点打印 steps，不可达打印 Impossible。

代码建立思路：这题基本套 graph.md 的“不同代价迷宫 Dijkstra”。现成部分是 heapq 小根堆、dist 二维数组、每次弹出当前最短时间状态、四方向松弛。新写/修改的是题目字符含义：起点是 r，终点是 a，墙不能进，进入 x 的代价是 2，进入其他可走格代价是 1。组合方式就是把网格当作隐式带权图，不显式建边，用 Dijkstra 在扩展邻格时即时计算边权。

这题和 md 的“解题思路二”高度一致。它最核心的判断是：普通 BFS 按步数扩展，但本题按总时间最小扩展。

4.10 57 连接所有点的最小费用

资料依据

md 依据：Prim 的 heapq 对象图版本、完全图概念、MST 定义。

本题实际代码方法

你的实际代码：用 ast.literal_eval 读 points；先为每对点计算曼哈顿距离并加入对象图，包含双向边；再调用 prim(g,g.getVertex(0))。Prim 中用 inTree 防止重复加入，用 distance 记录当前连接代价。

代码建立思路：这里用了 graph.md 的对象图 Prim/heapq Prim 思路。现成部分是 Vertex/Graph 存边、Prim 用堆维护“把某个新点接入当前树的最小代价”、用 inTree 跳过已经加入 MST 的点。你新写的部分是把点坐标转成完全图：两层循环枚举每一对点，计算曼哈顿距离，并作为无向边加入图。最后组合为：输入坐标 → 显式建立完全带权图 → 从 0 号点运行 Prim → 累加每次接入新点的最小边权。

这里也要校订：你的代码不是“不建完全图边”的简化 Prim，而是先显式建了完全图对象图，再跑 Prim。学案应讲这个实际方法，同时提醒它与 md 的 Prim 对象图版本最相近。

5 横向对比：md 模板如何被你实际改动

md 模板/概念	对应作业题	你实际做出的改动
前序 + 中序建树	22	保留 Buildtree，新增 EOF 多组输入和后序输出。
二叉树节点与遍历	23、50、49	23 增加按下标解析括号；50 增加 . 空节点和 nonlocal idx；49 从编号输入连接对象。
层次建树	54	用 # 建空节点，再上下界验证 BST。
手写堆	26	比较方向改成小顶堆；按操作流插入/删除。
heapq Huffman	28	基本照最小 WPL 模板，多组输入。
左儿子右兄弟转换	27	先按 d/u 建普通树，再调用 convert，最后求两种高度。
词梯桶建图 +BFS	单词序列	从输入列表建桶；把 start/end 加入词表；输出边数 +1。

普通迷宫 BFS	仙岛求药	起点 @, 终点 *, 墙 #, 0 0 结束。
状态 BFS	变换的迷宫	第三维从“剩余能力”变成“时间模数”。
不同代价迷宫	拯救行动	完全对应, 进入 x 代价 2。
Dijkstra		
最优资源状态思想	打怪救公主	用 max_blood 而不是布尔 visited。
拓扑排序队列	拓扑排序	队列改最小堆, 保证编号小优先。
Prim 邻接矩阵	丛林中的路	字母转编号, 矩阵存边, 堆式 Prim。
Dijkstra 对象图	兔子与樱花	每个查询 reset 后跑无优先队列 Dijkstra, 再 pred 回溯。
Dijkstra + DFS 自	A Walk Through the	从 2 跑 Dijkstra, 再按距离下降 DFS 计数。
行组合	Forest	
Prim 对象图	连接所有点最小费用	先显式建完全图, 再用对象图 Prim。

6 graph.md 中有具体代码的算法清单与题目映射

6.1 graph.md 中实际给出代码的算法/模板

这里只统计 graph.md 中出现了具体 Python 代码的算法或可直接复用的模板；只有文字说明、没有代码实现的内容不列入“有代码算法”。例如 Kruskal 在 graph.md 中只有文字定义和图示，没有给出并查集代码，因此不放入本表的“有代码算法”清单。

代码算法/模板	核心用途
Graph/Vertex 邻接表模板	建立邻接表图：用对象保存顶点、邻接点、边权；后续词梯、Dijkstra、Prim 都能套用。
Graph 邻接矩阵模板	建立邻接矩阵图：用二维矩阵存边，适合点数较小、需要按编号查边权的图。
词梯桶建图	建立“只差一步”的单词图：把只差一个字母的单词放入同一通配桶，再在桶内连边。
BFS 图遍历	按最少边数分层遍历：用颜色、距离、前驱和队列，从起点一层一层扩展。
BFS 路径回溯	恢复 BFS 找到的路径：沿 pred 从终点回溯到起点。
邻接矩阵 BFS	在矩阵图上做 BFS：把 BFS 模板改成通过矩阵取邻居。
数轴 BFS	在一维状态空间找最少步数：状态是一维位置，转移为 $x-1, x+1, 2*x$ 。
普通迷宫 BFS	在等代价迷宫中找最短步数：状态是 $(x, y, steps)$ ，二维 visited，四方方向扩展。
剩余能力状态 BFS	在带额外限制的迷宫中找最短步数：状态加入 remaining，判重数组变为三维。
不同代价迷宫 Dijkstra	在不同格子代价的迷宫中找最小总代价：普通格代价 1，守卫格代价 2，用 heapq 按总代价扩展。
骑士周游建图	把棋盘问题转成图问题：把棋盘格转成图顶点，把马走日合法移动转成边。
骑士周游 DFS 回溯	搜索覆盖所有棋盘格的路径：用 DFS、颜色标记、路径栈尝试走完整个图。
Warnsdorff 优化	减少骑士周游回溯次数：优先走后续可选位置最少的点。
通用 DFS	深度优先遍历整张图：用发现时间、完成时间、颜色、前驱记录搜索过程。
非递归 DFS	用栈实现深度优先遍历：用显式栈模拟递归 DFS。
城堡 DFS 染色	给连通区域编号并统计面积：用 DFS 给连通房间染色，统计房间数和最大面积。
回溯计数	枚举所有可行走法：走过格子标记，递归枚举方案，返回后撤销标记。

算 24 回溯	枚举数字合并顺序和运算：每次合并两个数，递归判断能否得到 24。
拓扑排序	给有依赖关系的任务排合法顺序：统计入度，用队列反复输出入度为 0 的点。
AOE 关键路径	找工程网络中的关键活动：在 DAG 上求最早/最晚发生时间和关键活动。
Dijkstra 优先队列版	求非负权图单源最短路径：用优先队列/decreaseKey 思想维护当前最短距离。
Dijkstra 无优先队列版	求小规模非负权图单源最短路径：每轮扫描未访问点中距离最小者。
Floyd	求所有点对之间的最短路径：维护 dist 和 prev，逐个中转点松弛。
Prim 对象图版	建立最小生成树：在对象图上构造 MST，使用优先队列和前驱。
Prim heapq 版	建立最小生成树：用 heapq 替代 decreaseKey，用 inTree 跳过旧状态。
Prim 邻接矩阵版	在矩阵图上建立最小生成树：用堆选择当前能连接到树的最小边。

6.2 作业题目与 graph.md 代码算法之间的映射

本表只映射 PDF 作业中属于图/搜索/MST/最短路的题；树和堆题主要对应 tree.md，不放在本节。

PDF 作业题目	对应 graph.md 有代码算法	本题实际采用方式
1 单词序列	词梯桶建图 + BFS 图遍历	你的代码基本沿用 buildGraph、Vertex/Graph、BFS；改为从输入列表建词表，并输出 distance+1。
2 仙岛求药	普通迷宫 BFS	你的代码用 deque、二维 visited、四方向扩展；起点 @，终点 *。
3 变换的迷宫	剩余能力状态 BFS 的“状态升维”思想	graph.md 代码第三维是剩余能力；本题实际第三维是 time % K，墙在特定时间可过。
1 拓扑排序	拓扑排序 topoSort(G)	graph.md 用普通队列；你的代码改用 heapq 存入度为 0 的点，保证编号小优先，并输出 v 前缀。
2 丛林中的路	Prim 邻接矩阵版	你的代码使用 Graph_adj_mat + prim(G,0)；字母点名转编号后写入无向邻接矩阵。
3 A Walk Through the Forest	Dijkstra 优先队列版 + DFS/记忆化思想	graph.md 没有这道组合题；你的代码从顶点 2 跑 Dijkstra，再沿 dist 严格下降方向 DFS 计数。
4 打怪救公主	普通迷宫 BFS + 状态/资源影响后续的思想	graph.md 没有最大血量模板；你的代码把布尔 visited 改为 max_blood[x][y]，更高血量才重入队。
5 兔子与樱花	Dijkstra 无优先队列版 + pred 路径恢复	你的代码每个查询 reset 后跑 dijkstra_no_pq，再用 pred 回溯并格式化输出路径；不是 Floyd。
56 拯救行动	不同代价迷宫 Dijkstra	与 graph.md 守卫迷宫 Dijkstra 代码高度对应：x 代价 2，其他可走格代价 1。
57 连接所有点的最小费用	Prim heapq 对象图版	你的代码先显式建立完全图对象图，边权为曼哈顿距离，再用 heapq Prim 求总费用。

需要警惕

不要把“graph.md 里有 Floyd 代码”直接映射到“兔子与樱花”。虽然题型上 Floyd 也能做多源多查询，但你当前 PDF 和 Python 文件中的实际方法是无优先队列 Dijkstra。学案复习应以实际代码为准。

7 考前落地检查表

落地检查

写代码前按顺序问：

1. 我现在用的是 md 中哪个模板？它在本题中具体改了哪几个输入变量？
2. PDF/同名 Python 文件里实际方法是什么？是手写结构、对象图，还是库函数？
3. 题目答案是距离、时间、路径、数量、剩余资源，还是连通总代价？
4. `visited` 是布尔、三维状态，还是最优值数组？
5. 如果用了堆，堆里第一个元素代表什么：距离、边权、编号优先级，还是堆节点值？
6. 如果用了树递归，根从哪里来：前序开头、后序结尾、当前字符、层序队首，还是编号 1？
7. 输出口径是否和样例一致：不可达输出 0、-1、Oop!、Impossible，还是别的？

8 最该优先复习的错题组合

1. 实现堆结构 + **Huffman**：一个手写 BinHeap，一个用 `heapq`；不要混淆。
2. 丛林中的路 + 连接所有点：两个都是 Prim，但前者邻接矩阵显式边，后者对象图完全图。
3. 兔子与樱花 + **A Walk Through the Forest**：两个都用了 Dijkstra 思想，但前者为每次查询找路径，后者从家出发算距离后 DFS 计数。
4. 仙岛求药 + 变换的迷宫 + 拯救行动 + 打怪救公主：四个都是迷宫，但分别是普通 BFS、时间模数 BFS、Dijkstra、最大资源 BFS。
5. **22 + 53 + 50 + 23**：四个都是树输入解析，但根的位置和空节点处理完全不同。